MUMmer, NUCmer et LASTZ

Fondements algorithmiques, installation GitHub, CLI et cas d'usage

Table des matières

1 Vue d'ensemble **2**

2 Fondements algorithmiques **2**
- 2.1 MUMmer 3 : arbre de suffixes compact . . . 2
- 2.2 MUMmer 4 : tableau de suffixes (suffix array) et LCP . . . 3
- 2.3 NUCmer : seed–cluster–align . . . 3
- 2.4 LASTZ : seed-and-extend à la BLASTZ . . . 4

3 Installation GitHub (Ubuntu) **4**
- 3.1 MUMmer4 + NUCmer . . . 4
- 3.2 LASTZ 1.04.52+ . . . 5

4 Manuel CLI de référence (avec valeurs par défaut et images) **5**
- 4.1 mummer – moteur de matches exacts . . . 5
- 4.2 nucmer – alignements ADN . . . 6
- 4.3 Outils autour de NUCmer . . . 7
- 4.4 LASTZ – CLI . . . 7

5 Entrées / sorties : pourquoi pas les reads bruts ? **9**

6 Cas d'usage préférentiels **10**
- 6.1 Quand utiliser NUCmer (MUMmer4) . . . 10
- 6.2 Quand utiliser LASTZ . . . 10

7 Récap pratique **10**

1 Vue d'ensemble

MUMmer (et son composant NUCmer) et LASTZ sont des outils historiques pour comparer des séquences génomiques de grande taille (génomes complets ou grands contigs).

Ils reposent sur deux grandes familles d'algorithmes :

- **MUMmer 3/4 :** indexe la référence avec un arbre de suffixes (v3) ou un tableau de suffixes (v4), puis trouve des Maximal Exact Matches (MEMs) ou Maximal Unique Matches (MUMs) utilisés comme ancres pour construire des alignements globaux.
- **LASTZ :** implémente un schéma seed-and-extend de type BLASTZ : graines (seeds), extension sans gap en HSPs, chaînage, puis alignement local gapé avec matrices de substitution et pénalités de gaps.

Ces outils sont conçus pour des séquences assemblées (contigs, scaffolds, génomes de référence), pas pour du mapping massif de reads bruts.

2 Fondements algorithmiques

2.1 MUMmer 3 : arbre de suffixes compact

MUMmer3 s'appuie sur une implémentation compacte d'arbre de suffixes inspirée des travaux de Kurtz, permettant de traiter efficacement des génomes de plusieurs centaines de mégabases.

- **Arbre de suffixes :** On considère tous les suffixes de la séquence de référence (positions $1..n, 2..n, \ldots, n..n$). L'arbre de suffixes stocke ces suffixes dans un trie compressé : chaque chemin racine→feuille correspond à un suffixe, avec des arêtes étiquetées par des sous-chaînes.
- **Complexité théorique de construction et de stockage :** $O(n)$ en temps et en espace, avec un gros facteur constant mémoire (plusieurs fois la taille de la séquence).
- **Recherche de motifs :** Pour trouver toutes les occurrences d'un motif de longueur k, on descend l'arbre en suivant les lettres du motif ; on arrive à un nœud qui couvre toutes les feuilles (suffixes) où ce motif apparaît. Complexité : $O(k)$ pour naviguer dans l'arbre, puis sortie de toutes les occurrences.
- **MUMs / MEMs :**
  - MEM (Maximal Exact Match) : match exact qui ne peut pas être étendu à gauche/droite sans introduire un mismatch, mais qui peut apparaître plusieurs fois dans chaque génome.
  - MUM (Maximal Unique Match) : MEM qui est unique dans la référence et dans la requête.

  MUMmer3 sélectionne ces segments en parcourant l'arbre de suffixes pour la référence et en insérant les suffixes de la requête.
- **Clustering de matches :** Des programmes auxiliaires (gaps/mgaps) regroupent les matches exacts en clusters le long de diagonales approximativement constantes, ce qui forme des blocs de synténie utilisables comme alignements.
- **Limitation :** l'arbre de suffixes 32 bits consomme tellement de mémoire que des génomes giga-échelles (plantes très répétées, pan-génomes entiers) deviennent difficiles à traiter, d'où MUMmer4.

Image pour débutant

Imagine que le génome est un livre très long. L'arbre de suffixes, c'est comme un index ultra-détaillé qui contient toutes les phrases possibles du livre, organisées pour qu'on les retrouve rapidement.
MUMmer parcourt cet index pour trouver des phrases identiques entre deux livres (génomes). Les MUMs sont les phrases qui n'apparaissent qu'une seule fois dans chaque livre : ce sont les meilleurs repères pour aligner les histoires.

2.2 MUMmer 4 : tableau de suffixes (suffix array) et LCP

MUMmer4 remplace le suffix tree par un suffix array (SA) et des structures associées (LCP, etc.), ce qui diminue l'empreinte mémoire et supprime les limites de taille.

- **Suffix array :** SA est un tableau d'entiers contenant les positions des suffixes de la référence, triées lexicographiquement (ordre alphabétique des suffixes).
- Pour chercher un motif de longueur k, on fait une recherche dichotomique dans ce tableau pour trouver la fenêtre des suffixes commençant par ce motif (en $O(k \log n)$), puis on récupère toutes les occurrences correspondantes.
- **Implémentation MUMmer4 :** SA 48-bits → limite théorique de 2^{48} positions, donc des téra-bases de séquence.par.cse.nsysu.edu
- Mémoire ≈ 10× la taille de la séquence plutôt que 20–40× pour un suffix tree, selon les benchmarks publiés.par.cse.nsysu.edu
- **Algorithme global :**
  - Concaténer toutes les séquences de référence (multi-FASTA) en une séquence linéaire et construire SA + LCP (linear-time SA algorithm).
  - Pour chaque requête, balayer la séquence et pour chaque position, chercher tous les MEMs/MUMs d'au moins `-minmatch` via SA.
  - Transmettre les matches exacts à mgaps pour les regrouper en clusters cohérents (même diagonale approximative, écarts limités).
  - Convertir ces clusters en alignements pour NUCmer/PROmer ou en matches bruts pour mummerplot.
- En complexité, on reste en $O(n)$ ou $O(n \log n)$ pour la construction et $O(m)$ pour le balayage des requêtes, avec bonne scalabilité pratique.

Image pour débutant

Le suffix array, c'est comme si tu listais toutes les phrases du livre par leur position, puis que tu les triais par ordre alphabétique. Pour chercher une phrase, tu fais une recherche binaire dans cette liste triée.
C'est un énorme dictionnaire de suffixes : un peu plus simple qu'un arbre, mais tout aussi puissant.

2.3 NUCmer : seed–cluster–align

NUCmer est le moteur d'alignement ADN basé sur MUMmer ; il implémente un schéma seed–cluster–align pour des séquences très similaires (≥ 90–95 %).

- **Seed :** recherche de matches exacts (MUMs/MEMs) d'au moins `-minmatch` bases, avec unicité contrôlée par `-mum`, `-mumreference` ou `-maxmatch`.mummer4.github
- **Cluster :** agrégation des matches le long de diagonales avec contraintes `-maxgap`, `-diagdiff` et `-diagfactor`[][].

— **Align :** conversion des clusters en alignements .delta, extension autour des ancres, calcul des différences (SNPs/indels) via show-snps, métriques globales via dnadiff.[][][]

NUCmer est optimisé pour être quasi linéaire sur la taille des génomes à haute identité : plus les génomes sont proches, plus il est efficace.]

2.4 LASTZ : seed-and-extend à la BLASTZ

LASTZ est un aligner pairwise pour ADN conçu comme remplaçant de BLASTZ, largement utilisé pour les comparaisons inter-espèces (vertébrés, etc.).[][][]

— **Seeding :** Index de la cible : table de positions associant un mot (seed) à ses coordonnées dans le génome.] Seeds définies par `-seed` (pattern BLASTZ par défaut, ou matchL pour mot exact de longueur L).]

— **Extension gap-free → HSPs :** Pour chaque hit de seed, LASTZ étend sans gap dans les deux sens, avec un schéma x-drop contrôlé par `-xdrop` : on s'arrête si le score chute de plus de xdrop par rapport au maximum.] Les segments dont le score dépasse `-hspthresh` deviennent des HSPs (High-Scoring Segment Pairs).]

— **Chaining :** Les HSPs sont regroupés en chaînes colinéaires qui maximisent un score global, en pénalisant les variations de diagonale/anti-diagonale via `-chain=diag,anti`.]

— **Alignement gapé :** À partir de ces chaînes, LASTZ effectue un alignement local gapé (type Smith-Waterman) en utilisant une matrice de substitution (`-scores` ou `-match`) et des pénalités de gap `-gap=open,extend`.] Un seuil `-gappedthresh` (par défaut égal à `-hspthresh`) filtre les alignements gapés trop faibles.]

Image pour débutant

Pense à LASTZ comme à un lecteur qui cherche des motifs caractéristiques (graines) dans deux livres :
quand il en trouve un, il lit autour pour voir s'il s'agit vraiment du même passage (HSP), relie ces passages en une histoire continue (chaining), puis comble les trous en permettant quelques mots manquants ou différents (gaps).

3 Installation GitHub (Ubuntu)

3.1 MUMmer4 + NUCmer

Prérequis :

```
sudo apt update
sudo apt install -y \
  build-essential git autoconf automake libtool gettext \
  yaggo swig python3-dev ruby-dev libperl-dev
```

build-essential : gcc, g++, make, etc.
autoconf, automake, libtool, gettext, yaggo : nécessaires si tu compiles depuis le dépôt GitHub (pas depuis une archive release).
Les librairies dev Python/Ruby/Perl sont requises pour certains bindings.

Clonage & compilation :

```
cd ~/src
git clone https://github.com/mummer4/mummer.git
cd mummer
# Generer configure & co
autoreconf -fi
```

```
# Configuration (par ex. installation dans ~/.local)
./configure --prefix=$HOME/.local
# Compilation + tests
make -j4
make check     # optionnel
# Installation
make install
```

3.2 LASTZ 1.04.52+

Prérequis :

```
sudo apt update
sudo apt install -y build-essential git
```

Clonage & installation : D'après le README GitHub et la doc LASTZ

```
cd ~/src
git clone https://github.com/lastz/lastz.git
cd lastz
# Choisir un dossier d'installation
export LASTZ_INSTALL=$HOME/.local/bin
mkdir -p "$LASTZ_INSTALL"
# Compilation + installation
make           # construit lastz et lastz_D
make install   # copie les binaires dans $LASTZ_INSTALL
```

Il suffit ensuite d'avoir `$HOME/.local/bin` dans ton PATH.

4 Manuel CLI de référence (avec valeurs par défaut et images)

4.1 mummer – moteur de matches exacts

Usage :

```
mummer [options] <reference.fa> <query.fa ...>
```

Principales options (MUMmer3/4) :

Option	Défaut	Effet	Image pour débutant
`-l LEN` `-minmatch=LEN`	20 pb	Longueur minimale d'un match exact (seed)	Taille minimale d'une « phrase » commune que tu juges intéressante
`-mum`	off	MUMs (uniques dans ref & query)	Phrases qui n'apparaissent qu'une fois dans chaque livre
`-mumreference`	on (v4)	MUMs uniques dans la référence seulement	Phrases uniques dans le livre 1, mais pouvant être répétées dans le livre 2
`-maxmatch`	off	Tous les MEMs sans contrainte d'unicité	Tu notes toutes les phrases communes, même les banalités
`-b`	off	Cherche aussi sur le brin complémentaire	Tu lis aussi le livre à l'envers pour voir s'il correspond
`-c`	off	Trie les matches par coordonnées	Remet toutes les phrases communes dans l'ordre des pages

Plus tu baisses `-l` et plus tu te rapproches de `-maxmatch`, plus tu récupères d'ancres (sensibilité), mais plus les dotplots deviennent denses et plus le calcul est lourd.

4.2 nucmer – alignements ADN

Usage :

```
nucmer [options] <reference.fa> <query.fa>
```

Options clés (avec défauts, d'après scripts NUCmer3/4 + wrappers Galaxy).[]]

Option	Défaut	Effet technique	Image
`-p PREFIX`	requis	Préfixe des sorties (PREFIX.delta, etc.)	Nom de projet
`-minmatch=LEN` `(-l)`	20	Longueur min des matches MUM/MEM	Phrase min pour qu'on la regarde
`-mincluster=LEN` `(-c)`	65	Longueur totale min d'un cluster d'ancres	Taille min d'un paragraphe commun
`-maxgap=G`	90	Distance max entre deux ancres dans un cluster	Longueur max d'un « trou » dans un paragraphe
`-breaklen=B`	200	Longueur max d'un segment non aligné avant de couper	Nombre max de phrases qui ne se ressemblent plus avant d'abandonner
`-mum`	off	MUMs uniques dans ref & query	Repères ultra fiables, peu nombreux
`-mumreference`	on	MUMs uniques dans la ref	Par défaut NUCmer préfère des ancres stables côté ref
`-maxmatch`	off	Tous les MEMs comme ancres	Repères très nombreux, y compris dans les répétitions

Effet sur sensibilité / temps :

— Scénario rapide, propre (génomes très proches, tu veux juste la structure globale) : `-mumreference -minmatch=20 -mincluster=65 -maxgap=90` (paramètres par défaut raisonnables).

— Scénario très sensible (génomes un peu plus divergents / très fragmentés) : diminuer `-minmatch` (15–17), voire `-mincluster`, ou utiliser `-maxmatch` – au prix de temps CPU et de .delta énormes.

4.3 Outils autour de NUCmer

delta-filter

```
delta-filter [options] in.delta > out.delta
```

`-1` : meilleur alignement 1-vers-1 entre chaque paire de séquences.[][]
`-r / -q` : filtrage meilleur alignement par référence / par requête.
`-i x` : identité minimale (en %).
`-l LEN` : longueur minimale d'alignement.
Image : tu prends tous les alignements bruts et tu ne gardes que les meilleurs (comme choisir une seule traduction par phrase).

show-coords

```
show-coords -rcl in.delta > out.coords
```

`-r` : trier par référence.
`-c` : afficher %identité.
`-l` : afficher longueurs.]
Donne un tableau : start/end ref, start/end query, longueurs, %id, longueurs totales, couvertures.

show-snps

```
show-snps -Clr in.delta > out.snps
```

`-C` : ne rapporter que les SNPs non ambigus.
`-l` : lignes longues.
`-r` : coordonnées orientées référence.]

dnadiff

```
dnadiff -p PREFIX ref.fa qry.fa
```

Pipeline complet (NUCmer + filtres + stats) pour obtenir un rapport global de similarité entre deux génomes.[]]

4.4 LASTZ – CLI

Usage général :]

```
lastz target.fa[multiple] query.fa[multiple] [options]
```

Flags d'entrée / brins

Option	Défaut	Effet
`-strand=both`	both	Chercher sur brin + et − []
`-self`	off	Utiliser target comme query (auto-alignement) []
`-ambiguous=n[,penalty]`	off	Traiter N comme base ambiguë (au lieu de séparateur) []
`-ambiguous=iupac[,penalty]`	off	Autoriser les codes IUPAC (S, R, Y, etc.) [][]

Seeding / HSPs

Option	Défaut	Effet
`-seed=PATTERN`	HOXD70	Forme des graines (motif) [][]
`-step=N`	1	Pas entre deux positions testées (1 = chaque base) []
`-[no]gfextend`	on	Activer l'extension gap-free pour former des HSPs []
`-hspthresh=SCORE`	3000	Score minimal pour retenir un HSP []
`-exact=L`	off	Au lieu de hspthresh, chercher des matches exacts d'au moins L []
`-inner`	(complexe)	Seuil interne pour interpolation d'HSPs []

seed = pochoir qu'on applique sur les deux textes ; plus il est strict, plus il faut une vraie ressemblance pour déclencher une graine. `-step` = tous les combien de caractères tu déplaces le pochoir. `-hspthresh` = note minimale pour que la graine élargie soit considérée comme un passage intéressant.

Chaînage / alignements gapés

Option	Défaut	Effet
`-[no]chain`	on	Activer/désactiver le chaînage des HSPs []
`-chain=diag,anti`	diag=12, anti=32	Pénalités de diagonale/anti-diagonale []
`-[no]gapped`	on	Activer/désactiver les alignements avec gaps []
`-notrivial`	off	Supprimer l'alignement trivial en self-alignement []

*C*haînage = relier les petits paragraphes en une histoire cohérente. Les pénalités diag/anti contrôlent à quel point la ligne peut « serpenter ».

Scores, gaps, xdrop / ydrop

Option	Défaut	Effet
`-scores=FILE`	HOXD70	Matrice de substitution []
`-match=R,P`	off	Score simple +R/$-P$ []
`-gap=open,extend`	400,30	Pénalités ouverture/extension de gap []
`-xdrop=VAL`	10 * sub[A,A]	Seuil x-drop pour extension sans gap []
`-ydrop=VAL`	open+300ext.	Seuil y-drop pour alignements gapés []

gap = prix à payer chaque fois qu'on suppose une insertion/délétion dans l'un des textes. xdrop / ydrop = limite de patience : si on accumule trop de pénalités, on arrête.

Filtres d'alignement

Option	Défaut	Effet
`-identity=min..max`	0..100	Filtrer par % identité []
`-coverage=min..max`	0..100	Filtrer par % couverture de la requête []
`-[no]entropy`	on	Filtre d'entropie pour éviter les régions trop répétées []
`-masking=count`	0	Masquer les positions cibles touchées > count fois []

identity = tu exiges que deux passages se ressemblent à au moins X % pour les considérer comme la même idée.

Formats de sortie

Option	Défaut	Effet
`-output=FILE`	stdout	Fichier de sortie []
`-format=TYPE`	lav	Format (lav, axt, maf, cigar, general, text, rdotplot) []
`-rdotplot=FILE`	off	Dotplot au format R []

`format=mapping` est un raccourci pratique pour un format tabulaire de mapping (nom1, start1, end1, nom2, strand2, start2, end2, identité, couverture, CIGARx-).]

5 Entrées / sorties : pourquoi pas les reads bruts ?

Types d'entrée

- **MUMmer / NUCmer :** attend FASTA ou multi-FASTA (ADN ou AA pour PROmer).[][] Pas de FASTQ : on considère que les séquences sont déjà assemblées ou au moins longues (contigs, scaffolds, chromosomes).
- **LASTZ :** accepte FASTA, FASTQ, nib, 2bit, hsx pour target et query.[] Peut techniquement aligner des reads, mais ne tient pas compte des scores Phred et n'est pas optimisé pour des milliards de petites séquences.

Pourquoi ce n'est pas l'outil pour les reads bruts

- Ils utilisent des scores simples et déterministes (match/mismatch, coût de gap) sans modéliser la qualité des bases.

— Leur logique de seeds (MUMs de $\geq$ 20 bp, seeds de type BLASTZ) suppose qu'il existe de longues régions quasi exactes entre les séquences – ce qui est vrai pour des génomes proches, beaucoup moins pour des reads courts et bruyants.
— Les aligners modernes pour reads (BWA-MEM, Bowtie2, minimap2-sr) emploient des FM-index, des heuristiques de seeding très denses, et renvoient des MAPQ calibrés ; ce n'est pas le cas ici.[]]

Formats de sortie

— **MUMmer/NUCmer :**
  — `.mums` / `.mum` : matches exacts (mummer).
  — `.delta` : alignements pour NUCmer/PROmer, interrogation via show-coords, show-snps, dnadiff.[]
  — `.coords`, `.snps`, `.report` → interprétation facile dans R, Python ou tableur.[]]
— **LASTZ :**
  — `lav` : format historique BLASTZ.
  — `axt` : alignements pairwise (courant dans les pipelines UCSC).
  — `maf` : format multi-séquence.
  — `general/mapping` : formats tabulaires orientés analyses custom.[]]

6 Cas d'usage préférentiels

6.1 Quand utiliser NUCmer (MUMmer4)

— **Synténie entre souches très proches d'une même espèce ($\geq$ 95 % d'identité) :** Deux génomes bactériens, deux souches de levure, humain vs humain/chimpanzé.[]] On obtient des dotplots clairs, des blocs de synténie, un comptage de SNPs/indels détaillé via show-snps ou dnadiff.[]]
— **Comparaison assemblage vs référence :** Évaluation d'un nouvel assemblage (contigs/scaffolds) par rapport à un génome de référence : intégrité, réarrangements, gaps, erreurs d'assemblage.]
— **Pan-génomique :** Comparaison rapide de dizaines/centaines de génomes proches contre une référence unique pour extraire core/ accessory genome.]

6.2 Quand utiliser LASTZ

— **Comparaison inter-espèces divergentes :** Alignements de génomes de vertébrés (humain–souris, oiseau–reptile, etc.), où des MUMs de $\geq$ 20 bp deviennent rares mais où il reste des régions conservées détectables via seeds/HSPs.[]]
— **Annotation comparative & chains/nets :** LASTZ est la brique de base des pipelines UCSC, TOGA, MultiZ : sortie en AXT/LAV/MAF, chaînage, puis construction de nets et d'alignements multi-espèces.[]]
— **Recherche de régions non codantes conservées (CNS) :** En baissant `-seed`, `-hspthresh` et en filtrant sur `-identity/-coverage`, on cible des blocs courts mais très conservés dans des régions régulatrices.]

7 Récap pratique

— Pour des génomes très proches (souches d'E. coli, levures, humain vs chimpanzé, assemblage vs référence) :
→ **NUCmer/MUMmer4** avec paramètres proches des valeurs par défaut (`-mumreference, -minmatch=20, -mincluster=65, -maxgap=90`).

— Pour des comparaisons inter-espèces (humain–souris, humain–rat, vertébrés divers) ou des workflows UCSC/TOGA :
→ **LASTZ**, en ajustant `-seed, -hspthresh, -gap` et en utilisant des formats axt, maf ou general/mapping.

Dans tous les cas : Données attendues : génomes ou contigs, pas des millions de reads bruts. Utiliser MUMmer pour les dotplots et les statistiques de différences, LASTZ pour les alignements plus fins et l'intégration dans des pipelines multi-espèces.